Exploring ARM Prototype Hardware

University of Kent

By: Leandro J. Gomes Baroca (lb676@kent.ac.uk) Supervised by: Mark Batty (M.J.Batty@kent.ac.uk)

Project Aim

The goal of the Project is to explore *CHERI capabilities* in C, essentially testing the behaviours of the new capability features via *litmus testing*. The reason to test these *capabilities* is to observe if the theoretical function of the *capability system* matches the results of the *litmus testing*, such as *CHERI* returning error where an instance with a low level language (such as C) run on the same machine (without *CHERI*) does not return such an error.

What is the Prototype?

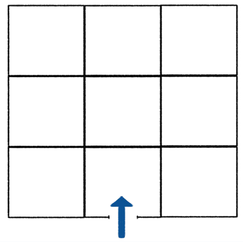
Morello & CHERI

Morello is a prototype "System on Chip" board currently being developed by ARM, to be released in Q3 or Q4 of 2021. This board contains an ARMv8-A processor with an implementation of the *CHERI* protection model, along with its own Graphics Processing Unit and Memory Subsystem. This system is planned to implement a research experiment on the implementation of *CHERI* within both software and hardware

Capability Hardware Enhanced RISC Instruction, or **CHERI** is an implementation of already existing ISAs (Instruction-Set Architectures) to processors by implementing a new capability architecture design to give safety design features to the memory of the hardware and introducing the implementation of scalable *compartmentalisation* of the software.

What is Compartmentalisation?

Compartmentalisation is a design strategy in *CHERI* implemented as a safety feature against software vulnerability, as one compartment cannot affect the rest of them. Essentially, it is the segregation of the memory of the *Morello* architectural model into individual compartments. So if it was breached by an attack, the memory in its entirety wouldn't be compromised. Contrary to a standard memory architecture, where the entire memory, or even computer system, would be compromised if it was breached.



Capabilities & Pointers

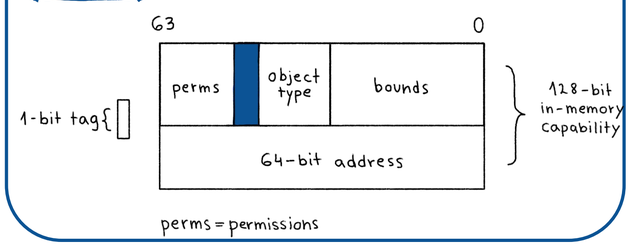
Capabilities in the *CHERI* design is a safety feature to protect **pointers** in virtual address spaces by protecting the integrity using additional metadata. This metadata includes: **Tags**, **Bounds**, **Object Type** and **Permissions**.

Tags are 1-bit units of memory, not stored in addressable memory, to track the validity of the capability. If tags are invalid, the capability cannot load, store, instruction fetch, etc. Tags also guarantee pointer behaviour when running unreliable programmes.

Bounds allow pointers to be out of bounds temporarily, but it has to return back in-bounds. This prevents exploitation of out of bounds execution and any buffer overflows. Bounds, in a pure-capability runtime space, are limited to the size of the memory allocated to it.

Object Types are metadata that allows additional software metadata to be linked with *sealed capabilities*, intended only to be used in combination without separation.

Capability Pointer Structure



Permissions give the capability a set purpose on what its usage is, by granting permission to load and store data via capabilities, but capabilities with permissions set to load cannot be used to store. In *CHERI*, permissions can be related to memory protection, while some permissions are defined in the software (i.e. to authorise modifications on virtual memory).

Description of Methodologies

Litmus Testing

The **litmus tests** programs are written (in C) to purposefully be invalid according to the guarantees of the *CHERI* architecture. These tests will be compiled on the x86 and cross compiled on *CHERI*, to then compare the behaviour when executed on x86 and *CHERI* through *QEMU*. The *CHERI* tests, in theory, should throw runtime errors, and the x86 ones should behave differently, possibly even running despite the coding errors they contain.

For instance, in the C code example on the right-hand side, it tests by memory allocating the size of a "uint64" pointer into variable "p", where *CHERI* would set a bound to that pointer, meaning "sizeof" will become the bounds. Then a "uint64" type with the variable "q" will be created, which will set the value of the pointer to be "p" plus itself again. This shouldn't be allowed by *CHERI* as the pointer is now out of bounds.

Litmus Test C Example

```c
#include <stdio.h>
#include <stdint.h>

int main (void) {
    uint64_t* p = malloc(sizeof(uint64_t));
    *p = 0;

    printf("The size of a unit64 is %i\n", sizeof(uint64_t));

    uint64_t* q = p + 1;
    *q = 42;

    printf("The value of the next unit64_t is %i\n", *q);

    return 0;
}
```

QEMU & CheriBSD

QEMU is an open-source processor emulator for virtualizing hardware, which in this case is the *Morello* board. The specific version of *QEMU* used to carry out the litmus tests is a forked version that provides support of *Morello*, by using *CHERI-RISC-V* and *CHERI-MIPS* (the example to the right-hand side is executed using *CHERI-MIPS*). With *CHERI-MIPS* being the stable version and *CHERI-RISC-V* being the development version.

CheriBSD is a *CHERI* adaptation of the open-source operating system called "FreeBSD", that is currently in prototype stages. CheriBSD has a hybrid implementation of the *CHERI* capability model and MMU-based virtual memory, along with C language implementations. The purpose of this prototype operating system is to implement memory protection with a standard operating system architecture.

Ubuntu VM Example x86 vs QEMU

Running the code on a standard x86 machine:

```
leandro@leandro-VirtualBox:~$ ./litmus1
The size of a unit64 is 8
The value of the next uint64_t is 42
```

Running the code on a QEMU-CHERI machine:

```
Feb 10 20:53:18 cheribsd-mips64-purecap kernel:
USER_CHERI_EXCEPTION: pid 659 tid 100053 (litmus
1), uid 0: CP2 fault (type 0x32)
Feb 10 20:53:18 cheribsd-mips64-purecap kernel:
Process arguments: ./litmus1
In-address space security exception (core dumped
)
```

Results

The **results** are based on testing the guarantees that *CHERI* provides, which are security features on bounds of a capability. So far, the **results** of litmus testing conclude:

- **Any attempts of dereferencing a capability that is currently out of bounds will throw exceptions.**
- **Any attempt at separating the capability address from the capability metadata assigned to it will throw exceptions.**
- **Any attempts at performing accesses to a capability that isn't authorized by the permissions will throw exceptions.**

Currently, there are still ongoing test attempts (may be discarded if impossible) to confirm if:

- **Any attempts of dereferencing capabilities (to load, store or fetch) without valid tags will throw exceptions.**
- **Any attempts at dereferencing or editing sealed object capabilities associated with software metadata will throw exceptions.**